# Manual OpenFoam

April 7, 2026

## 1 IncompressibleVoF

Para facilitarnos el uso de `incompressibleVoF`, importaremos el ejemplo de "Breaking of a Dam". Al abrir la terminal, cambiamos al siguiente directorio:

```
cd run/user/OpenFOAM
```

Luego escribimos:

```
cp -r $FOAM_TUTORIALS/incompressibleVoF/damBreakLaminar .
```

### 1.1 Por definir

Primero, revisemos los archivos que obtenemos al copiar la carpeta.

El primer archivo que vamos a editar es `system/blockMeshDict`. En este archivo se definen tres partes: `vertices`, `blocks` y `boundary`.

- En `vertices` colocamos todos los vértices de las caras de nuestro *mesh* con el formato ($x$ $y$ $z$). Al terminar de colocar los puntos, estos quedarán enlistados comenzando por el 0 según su aparición en el código.

- En `blocks` definimos los bloques mediante los puntos. La sintaxis es:

  ```
  hex (p1 p2 p3 ... p8) (a b c)
  ```

  Los 8 puntos deben colocarse de manera que si existe una arista entre dos puntos, estos aparezcan consecutivos en la lista. Los valores $a$, $b$ y $c$ indican la división del bloque en cada dirección.

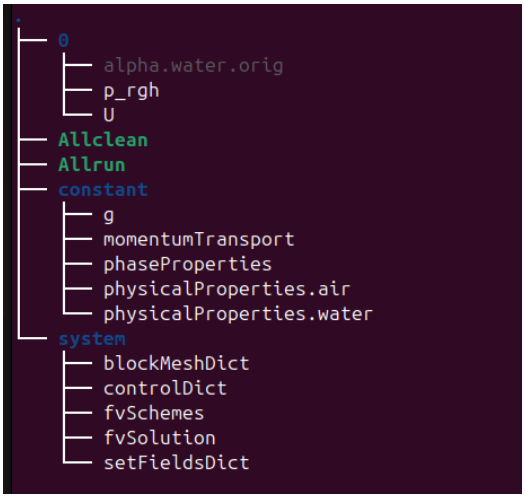



Figure 1: Archivos de `damBreakLaminar`

- En `boundary` definimos las paredes de los bloques. Los tipos más comunes son:
  - `type wall`: pared sólida dentro del dominio.
  - `type patch`: permite entrada y salida de fluido; no aporta información sobre la pared.
  - `type symmetryPlane`: replica lo que ocurre en el plano de simetría.
  - `type empty`: útil para modelar en 2D o 1D, ya que el sistema no resuelve ecuaciones en esas direcciones.

Al determinar las paredes, podemos agrupar distintas caras si tienen el mismo tipo o alguna propiedad particular que será importante más adelante. A esto se le llama `boundaryField`.

Para verificar cómo quedó el *mesh*, usamos los comandos:

```
blockMesh
paraFoam
```

`blockMesh` compila el código y `paraFoam` abre un visualizador para ver el *mesh*.

Ahora editaremos las condiciones iniciales del sistema, que se encuentran en la carpeta `0`. Empecemos con el archivo `U`, que contiene las velocidades

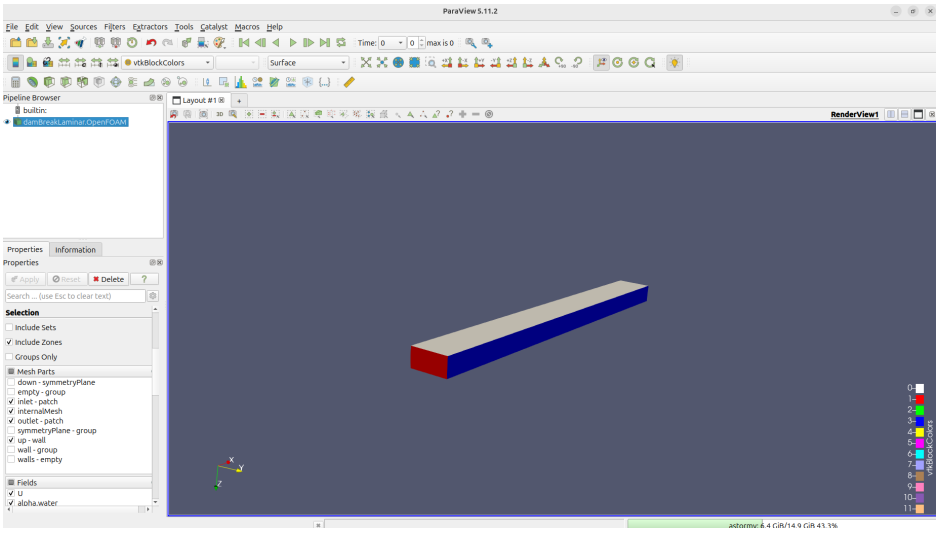



Figure 2: Modelo 3D del dominio

del agua en las caras del sistema (llamado `boundaryMesh`). Los tipos que se pueden asignar son:

- `fixedValue`: se asigna un valor fijo de velocidad mediante la línea `value uniform (vx vy vz)`.
- `zeroGradient`: la velocidad se obtiene de la celda vecina; se usa usualmente en las caras de salida.
- `symmetryPlane`: replica lo que ocurre en el plano de simetría.
- `noSlip`: produce paredes sin deslizamiento.
- `slip`: permite que el fluido resbale sobre la cara seleccionada.
- `empty`: no agrega ninguna condición.

También en la carpeta `0` tenemos el archivo `p_rgh`, que representa la presión hidrostática. Aquí podemos modificar los tipos de `boundaryField`. Los tipos son los mismos que en `U`, excepto `slip` y `noSlip`. Se añade además `fixedFluxPressure`, que es una presión fija determinada por el sistema.

Con esto solo quedaría revisar las propiedades físicas del aire y del agua. Estos archivos están en la carpeta `constant`. Podemos modificar la viscosidad (`nu`) y densidad (`rho`) del líquido y del aire en los archivos `physicalProperties.water`

y `physicalProperties.air`, respectivamente. También debemos modificar `phaseProperties`, donde definimos las fases del líquido con `phases (water air)` y el coeficiente de tensión superficial (`sigma`).

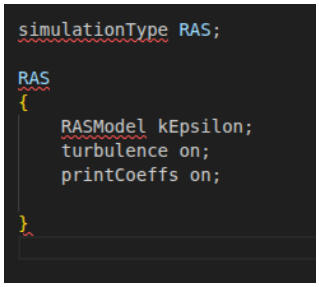
Los siguientes archivos que agreguemos o modifiquemos estarán relacionados con si el flujo es laminar o turbulento.

## 1.2 Flujo laminar

Para flujo laminar, como el ejemplo que estamos usando ya emplea este modelo, solo mostraremos dónde se define. Abrimos la carpeta `constant` y el archivo `momentumTransport`. Allí podemos ver la variable `simulationType` con el valor `laminar`.

## 1.3 Flujo turbulento

Para flujo turbulento, debemos modificar el archivo `constant/momentumTransport` y establecer `simulationType RAS` (Reynolds-Averaged Simulation). Debajo de esto indicamos que usaremos el modelo `kEpsilon` y que hay turbulencia.



```
simulationType RAS;

RAS
{
    RASModel kEpsilon;
    turbulence on;
    printCoeffs on;

}
```

Figure 3: Formato del archivo `RAS`

Además, debemos agregar los siguientes archivos en la carpeta `0`: `k`, `nut` y `epsilon`. Empecemos con `k`, donde se define la energía cinética turbulenta. En este archivo debemos especificar los `boundaryField`. Las opciones principales son:

- `fixedValue`: se asigna un valor fijo.
- `inletOutlet`: se usa cuando la dirección del flujo es desconocida.
- `symmetryPlane`

- `zeroGradient`
- `empty`

Luego revisamos `epsilon`, que representa la tasa de disipación de la energía cinética turbulenta, y tiene los mismos tipos que `k`. Finalmente, `nut` es la viscosidad turbulenta.

## 1.4 Agregar funciones de medición

En el archivo `system/controlDict`, al final podemos agregar funciones de medición mediante la definición de `functions` y `probes` de la siguiente manera:

```
functions
{
    probes
    {…
    }

}

DebugSwitches
{
    MULES           1;
}
```

Figure 4: Ejemplo de funciones de medición en `controlDict`

Dentro de `probes`, si queremos medir la presión, hacemos lo siguiente: primero definimos el campo que nos interesa (en este caso la presión, que está en el archivo `p_rgh`) y luego indicamos los puntos donde queremos medir mediante `probeLocations`.

## 1.5 Ejecución del programa

Para ejecutar el programa usamos:

```
probes
{
    type            probes;
    libs            ("libsampling.so");
    writeControl    adjustableRunTime;
    writeInterval   1;

    fields
    (
        p_rgh
    );

    probeLocations
    (
        (0.5 0.25 0)
        (0.5 0.25 10)

    );
}
```

Figure 5: Función de medición de presión con `probes`

`foamRun`

Para visualizar los resultados:

`paraFoam`

Una vez dentro de ParaView, cambiamos `Solid Color` por `alpha.water` y obtenemos una visualización similar a la siguiente:

Figure 6: Movimiento del fluido (campo `alpha.water`)